
App Python + Metrics + Ingress + Prometheus + Argo CD

Table of Contents

App Python + Metrics + Ingress + Prometheus + Argo CD.....	1
0 Principe du lab.....	2
1-Structure du projet.....	3
2 STEP 1 — Vérifier l’environnement.....	3
2.1 Cluster -- lancer k3d.....	3
2.2 Namespace monitoring.....	4
2.3 Prometheus actif.....	4
3 — Step2 -- Créer l’application Python.....	4
📄 app-code/app.py	5
4 STEP 3 — Docker.....	5
📄 app-code/Dockerfile	5
5 STEP 4 — Kubernetes manifests.....	7
📄 k8s/deployment.yaml	7
📄 k8s/service.yaml	8
📄 k8s/ingress.yaml	8
6 STEP 5 — ServiceMonitor (clé du problème).....	9
📄 k8s/servicemonitor.yaml	9
7 STEP 6 GIT.....	10
7.1 Initialiser Git local :.....	10
7.2 Créer le repo GitHub.....	11
7.3 Connecter le repo local à GitHub.....	11
8 STEP 7 — Argo CD.....	11
📄 app.yaml	11
9 STEP 8 — Déploiement.....	12
10 STEP 9 — Verifications.....	12

10.1 Pods.....	12
10.2 App.....	13
10.3 Metrics.....	13
11 STEP 10 — Vérifier Prometheus.....	14
11.1 port-forward.....	14
11.2 Onglet Targets.....	14
11.3. Onglet Graph.....	15
12 STEP 11. Test GitOps.....	15
12.1 Modifier un des fichiers :.....	16
12.2 Push :.....	16
12.3 Résultat :.....	16
13 STEP 12. Grafana.....	16
13.1 Lancer grafana.....	16
13.2 Résultat dans grafana :.....	17
14 Annexe 1 :.....	17
14.1 Méthode standard.....	17
14.1.1 STEP 1 — Installer Helm (si pas déjà fait).....	18
14.1.2 STEP 2 — Ajouter le repo Prometheus.....	18
14.1.3 STEP 3 — Créer namespace.....	18
14.1.4 STEP 4 — Installer kube-prometheus-stack.....	18
14.1.5 STEP 5 — Vérifier.....	18
14.2 Alternative (moins recommandée).....	19

O Principe du lab

👉 Ce lab **suppose** :

- Kubernetes fonctionne
- Prometheus/Grafana **déjà installés** (lab précédent : <https://github.com/guihen01/argo-lab>)
- serviceMonitor installé (helm a installé : kube-prometheus-stack)
- voir Annexe 1 (détails pour l'installation des pré-requis) au cas ou un des utilitaires ne seraient pas installés ou bien non fonctionnels.

1-Structure du projet

👉 On sépare **app** et **infra**

```
app-observability-lab/
├── app-code/                # code Python
│   ├── app.py
│   └── Dockerfile
├── k8s/                     # manifests Kubernetes
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── ingress.yaml
│   └── servicemonitor.yaml
└── app.yaml                 # Argo CD
```

2 STEP 1 — Vérifier l'environnement

2.1 Cluster -- lancer k3d

K3d start

Vérifier si au moins 1 node est présent

```
kubectl get nodes
```

👉 au moins 1 node Ready

```
henri@Henri:~/labs$
henri@Henri:~/labs$ kubectl get nodes
NAME                STATUS    ROLES                  AGE   VERSION
k3d-henri-server-0 Ready     control-plane,master   46h   v1.31.5+k3s1
henri@Henri:~/labs$
henri@Henri:~/labs$
```

2.2 Namespace monitoring

```
kubectl get ns
```

```
henri@Henri:~/labs$  
henri@Henri:~/labs$ kubectl get ns  
NAME                STATUS   AGE  
argocd               Active   8h  
default              Active   45h  
kube-node-lease      Active   45h  
kube-public          Active   45h  
kube-system          Active   45h  
monitoring           Active   45h  
henri@Henri:~/labs$
```

👉 On doit voir :

```
monitoring  
argocd  
default
```

2.3 Prometheus actif

```
kubectl get pods -n monitoring
```

```
henri@Henri:~/labs$ kubectl get pods -n monitoring  
NAME                                                                 READY   STATUS    RESTARTS   AGE  
alertmanager-monitoring-kube-prometheus-alertmanager-0            2/2    Running   0           26h  
monitoring-grafana-5d9d6dcf85-jnbmz                                3/3    Running   0           26h  
monitoring-kube-prometheus-operator-77d477b5cb-r7d6p              1/1    Running   0           26h  
monitoring-kube-state-metrics-57fb56f896-j5gm6                    1/1    Running   0           26h  
monitoring-prometheus-node-exporter-mmfv8                         1/1    Running   2 (9h ago)  46h  
prometheus-monitoring-kube-prometheus-prometheus-0               2/2    Running   0           26h  
henri@Henri:~/labs$
```

👉 attendu :

```
prometheus-xxxx    Running  
grafana-xxxx       Running
```

⚠️ IMPORTANT 👉 Si Prometheus n'est PAS là : ➡️ STOP → relancer votre cluster / helm

3 — Step2 -- Créer l'application Python

```
mkdir app-observability-lab  
cd app-observability-lab
```

```
mkdir app-code  
cd app-code
```



app-code/app.py

```
from flask import Flask
from prometheus_client import Counter, generate_latest

app = Flask(__name__)

REQUESTS = Counter('app_requests_total', 'Total HTTP Requests')

@app.route("/")
def home():
    REQUESTS.inc()
    return "Hello from Observability Lab 🚀"

@app.route("/metrics")
def metrics():
    return generate_latest(), 200, {'Content-Type': 'text/plain'}

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

4 STEP 3 — Docker



app-code/Dockerfile

```
FROM python:3.9-slim

WORKDIR /app
COPY app.py .

RUN pip install flask prometheus_client

CMD ["python", "app.py"]
```

Explication Docker :

Ligne	Explication
FROM	image de base
WORKDIR	dossier interne
COPY	copie code
RUN	install dépendances
CMD	lance l'app



Build & push

Il faut utiliser votre compte (username) de docker et remplacer <dockerhub> par votre username docker dans la commande ci-dessous :

Cette commande Construis une image (container) Docker en utilisant le fichier **Dockerfile du dossier courant**

```
docker build -t <dockerhub>/python-app:1.0 .
docker push <dockerhub>/python-app:1.0
```

par exemple, dans mon cas :

- (1) : docker build -t guihen01/python-app:1.0 .
- (2) : docker push guihen01/python-app:1.0

Kubernetes ne peut pas utiliser une image locale ➡ Il devra par la suite faire un **pull depuis un registry** (Docker Hub ici). Donc une fois l'image générée (1), docker fera un push de l'image vers son registre (repository) local (2).

Partie Signification

guihen01 mon compte Docker Hub
python-app nom de l'image
1.0 version (tag)

Résultat de la commande : docker push guihen01/python-app:1.0

```
henri@Henri:~/labs/app-observability-lab/app-code$
henri@Henri:~/labs/app-observability-lab/app-code$ docker push guihen01/python-app:1.0
The push refers to repository [docker.io/guihen01/python-app]
fc7443084902: Pushed
3ba2708c2cba: Pushed
38513bd72563: Pushing [=====] 29.78MB/29.78MB
170f1654d404: Pushed
ea56f685404a: Pushed
6546554759ca: Pushed
b3ec39b36ae8: Pushed
```

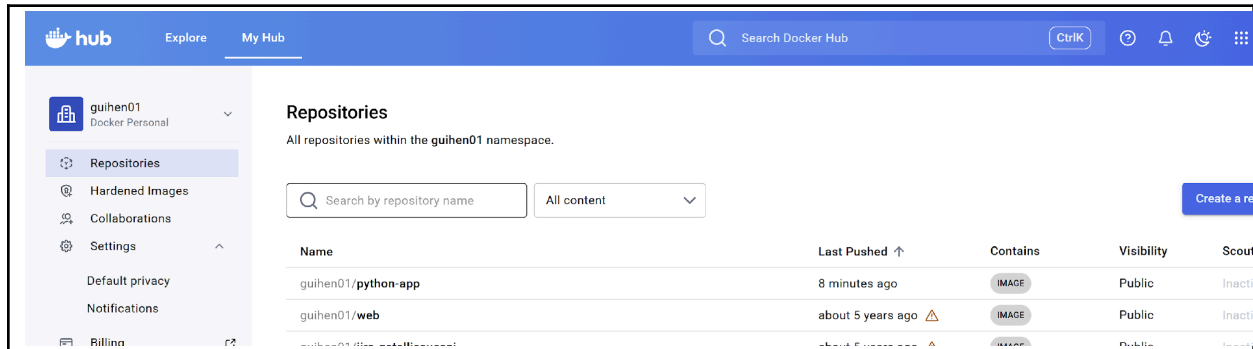
Vérifier :
Docker images

```
henri@Henri:~/labs/app-observability-lab/app-code$
henri@Henri:~/labs/app-observability-lab/app-code$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
guihen01/python-app	1.0	a689f75d247f	9 minutes ago	203MB
ubuntu	22.04	962f6cadeae0	2 weeks ago	117MB
ubuntu	latest	84e77dee7d1b	3 weeks ago	117MB
gcr.io/k8s-minikube/kicbase	<none>	eb4fec00e8ad	2 months ago	1.94GB
gcr.io/k8s-minikube/kicbase	v0.0.50	b97074569ae9	2 months ago	1.94GB
public.ecr.aws/lambda/python	3.11-rapid-x86_64	30eee7615ba6	3 months ago	1.16GB
public.ecr.aws/lambda/python	3.11-x86_64	6f0f64c34bed	3 months ago	1GB

Autre vérification possible :
À partir de dockerhub ,(si vous l'Avez installé)

Dans un browser taper l'URL de DockerHub : <https://hub.docker.com>
Dasn mon cas : <https://hub.docker.com/repositories/guihen01>



5 STEP 4 — Kubernetes manifests

```
cd ..  
mkdir k8s  
cd k8s
```



k8s/deployment.yaml

```
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: python-app  
spec:  
  replicas: 2  
  selector:  
    matchLabels:  
      app: python-app  
  template:  
    metadata:  
      labels:  
        app: python-app  
    spec:  
      containers:  
        - name: python-app  
          image: <dockerhub>/python-app:1.0  
          ports:  
            - containerPort: 5000
```



k8s/service.yaml

```
apiVersion: v1
```

```
kind: Service
metadata:
  name: python-app
  labels:
    app: python-app    # 🔥 CRUCIAL
spec:
  selector:
    app: python-app
  ports:
    - name: http        # 🔥 CRUCIAL
      port: 80
      targetPort: 5000
```



k8s/ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: python-app
spec:
  rules:
    - host: python.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: python-app
                port:
                  number: 80
```

🧠 Explication

👉 Permet accès navigateur :

http://python.local

⚠️ nécessite :

Ingress Controller installé

entrée /etc/hosts

6 STEP 5 — ServiceMonitor (clé du problème)

uniquement si kube-prometheus-stack

On peut vérifier si kube-prometheus-stack est installé avec la commande :
kubectl get crd | grep servicemonitor

On voit ici dans notre lab que cet utilitaire a été installé, car on voit :
servicemonitors.monitoring.coreos.com

```
henri@Henri:~/labs/app-observability-lab/k8s$ kubectl get crd | grep servicemonitor
servicemonitors.monitoring.coreos.com 2026-04-27T02:03:25Z
henri@Henri:~/labs/app-observability-lab/k8s$
henri@Henri:~/labs/app-observability-lab/k8s$
```



k8s/servicemonitor.yaml

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: python-app
  labels:
    release: kube-prometheus-stack # ⚠ À adapter
spec:
  selector:
    matchLabels:
      app: python-app
  endpoints:
    - port: http
      path: /metrics
      interval: 15s
```

⚠ POINT CRITIQUE :

Vérifier le label Prometheus :

```
kubectl get servicemonitor -n monitoring --show-labels
ou mieux :
kubectl get servicemonitor -n monitoring --show-labels | grep release
```

```

henri@Henri:~/labs/app-observability-lab/k8s$
henri@Henri:~/labs/app-observability-lab/k8s$ kubectl get servicemonitor -n monitoring --show-labels | grep release
monitoring-grafana                2d13h   app.kubernetes.io/instance=monitoring,app.kubernetes.io/man
aged-by=Helm,app.kubernetes.io/name=grafana,app.kubernetes.io/version=13.0.1,helm.sh/chart=grafana-12.1.3,release=monito
ring
monitoring-kube-prometheus-alertmanager  2d13h   app.kubernetes.io/instance=monitoring,app.kubernetes.io/man
aged-by=Helm,app.kubernetes.io/part-of=kube-prometheus-stack,app.kubernetes.io/version=84.1.2,app=kube-prometheus-stack-
alertmanager,chart=kube-prometheus-stack-84.1.2,heritage=Helm,release=monitoring
monitoring-kube-prometheus-apiserver    2d13h   app.kubernetes.io/instance=monitoring,app.kubernetes.io/man
aged-by=Helm,app.kubernetes.io/part-of=kube-prometheus-stack,app.kubernetes.io/version=84.1.2,app=kube-prometheus-stack-
apiserver,chart=kube-prometheus-stack-84.1.2,heritage=Helm,release=monitoring
monitoring-kube-prometheus-coredns      2d13h   app.kubernetes.io/instance=monitoring,app.kubernetes.io/man
aged-by=Helm,app.kubernetes.io/part-of=kube-prometheus-stack,app.kubernetes.io/version=84.1.2,app=kube-prometheus-stack-
coredns,chart=kube-prometheus-stack-84.1.2,heritage=Helm,release=monitoring

```

👉 On verra un label du type :

release=kube-prometheus-stack

👉 On doit utiliser EXACTEMENT le même, ici on utilisera dans notre cas : monitoring

Donc, dans notre cas, on modifie le fichier : servicemonitor.yaml

```

1  apiVersion: monitoring.coreos.com/v1
2  kind: ServiceMonitor
3  metadata:
4    name: python-app
5    labels:
6      release: monitoring # ⚠️ À adapter
7  spec:
8    selector:
9      matchLabels:
10        app: python-app
11    endpoints:
12      - port: http
13        path: /metrics
14        interval: 15s
15

```

7 STEP 6 GIT

7.1 Initialiser Git local :

```

Cd app-observability-lab/
git init
git add .
git commit -m "initial commit - python app + k8s manifests + argocd"

```

7.2 Créer le repo GitHub

👉 Aller sur GitHub :

- New repository
- Name: app-observability-lab
- Public
- NE PAS cocher README (important)

👉 On obtiens une URL : <https://github.com/guihen01/app-observability-lab.git>

7.3 Connecter le repo local à GitHub

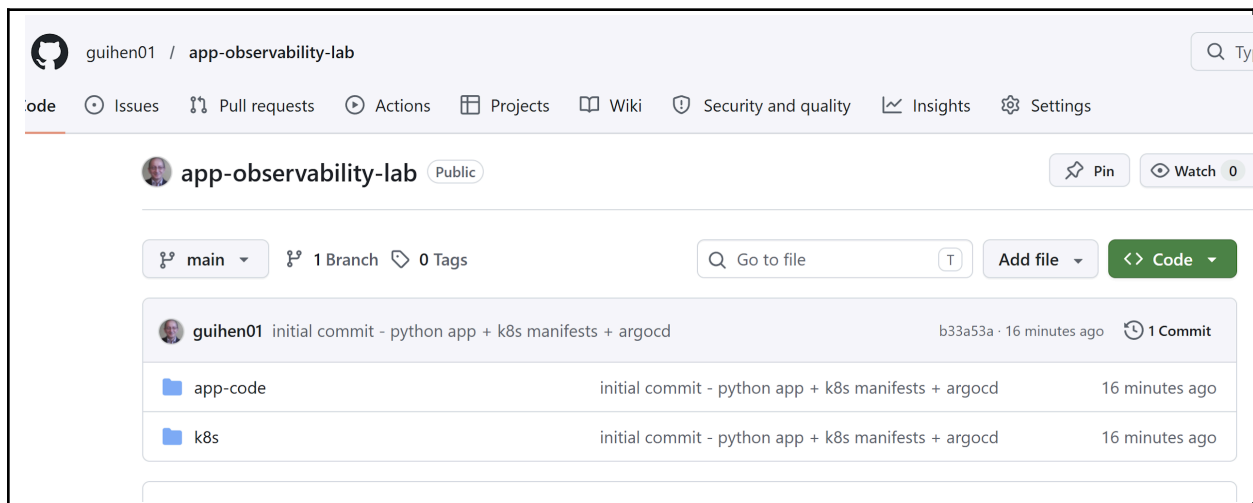
git branch -M main

git remote add origin https://github.com/guihen01/app-observability-lab.git

git push -u origin main

Résultat :

La structure locale est copiée sur le répertoire github :



8 STEP 7 — Argo CD

 **app.yaml**

Cd app-observability-lab/

Nano app.yaml

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
```

```
name: python-app
namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/guihen01/app-observability-lab
    targetRevision: main
    path: k8s
  destination:
    server: https://kubernetes.default.svc
    namespace: default
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

9 STEP 8 — Déploiement

```
kubectl apply -f app.yaml
```

```
henri@Henri:~/labs/app-observability-lab$ kubectl apply -f app.yaml
application.argoproj.io/python-app created
henri@Henri:~/labs/app-observability-lab$
henri@Henri:~/labs/app-observability-lab$
```

10 STEP 9 — Verifications

10.1 Pods

```
kubectl get pods
```

```
henri@Henri:~/labs/app-observability-lab$
henri@Henri:~/labs/app-observability-lab$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
demo-app-58bf5cfc8d-ckd28           1/1     Running   0           19h
nginx-7769f8f85b-75j6t              1/1     Running   0           19h
nginx-7769f8f85b-8j92t              1/1     Running   0           19h
nginx-7769f8f85b-zchgx              1/1     Running   0           19h
python-app-7d88cbcb74-5vkpt          1/1     Running   0           44s
python-app-7d88cbcb74-ssjh7          1/1     Running   0           44s
henri@Henri:~/labs/app-observability-lab$
henri@Henri:~/labs/app-observability-lab$
```

On constate que deux pods ont été créés (ceux dans le cadre rouge dans la figure ci-dessus)

10.2 App

Ajoute dans /etc/hosts :

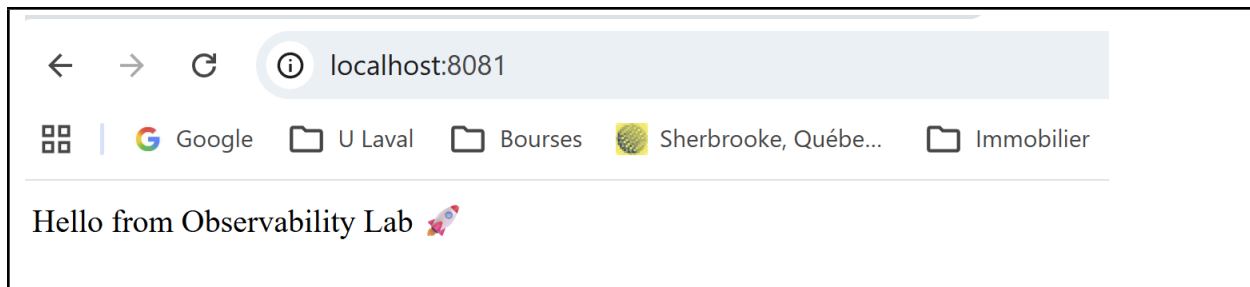
```
127.0.0.1 python.local
```

Puis :

```
kubectl port-forward svc/python-app 8081:80
```

👉 <http://localhost:8081>

Résultat :



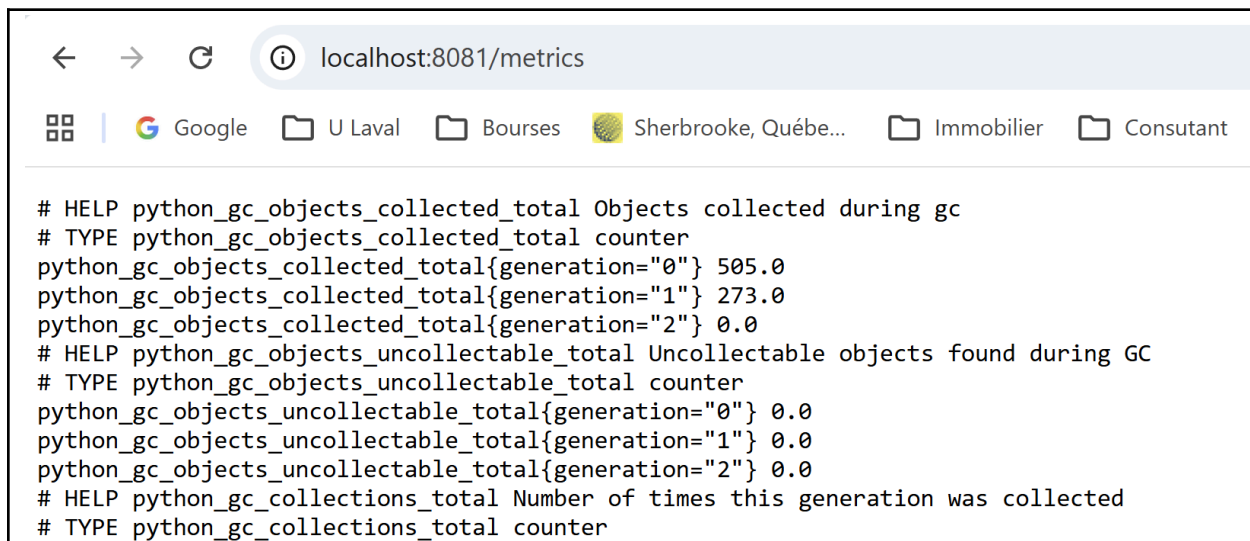
ton container fonctionne. ton Deployment est OK. ton Service est OK. le port-forward marche

👉 **Toute la chaîne app → Kubernetes fonctionne** ✓

10.3 Metrics

<http://localhost:8081/metrics>

👉 On doit voir du texte Prometheus



11 STEP 10 — Vérifier Prometheus

11.1 port-forward

Toujours faire une commande comme ci-dessous avant de faire une commande port-forward
kubectl get svc -n monitoring

```
henri@Henri:~/labs/app-observability-lab$ kubectl get svc -n monitoring
NAME                                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)                                AGE
alertmanager-operated              ClusterIP    None           <none>          9093/TCP,9094/TCP,9094/UDP            2d18h
monitoring-grafana                 ClusterIP    10.43.112.129  <none>          80/TCP                                2d18h
monitoring-kube-prometheus-alertmanager ClusterIP    10.43.11.209   <none>          9093/TCP,8080/TCP                    2d18h
monitoring-kube-prometheus-operator ClusterIP    10.43.155.55   <none>          443/TCP                                2d18h
monitoring-kube-prometheus-prometheus ClusterIP    10.43.191.172  <none>          9090/TCP,8080/TCP                    2d18h
monitoring-kube-state-metrics      ClusterIP    10.43.227.55   <none>          8080/TCP                                2d18h
monitoring-prometheus-node-exporter ClusterIP    10.43.105.211  <none>          9100/TCP                                2d18h
prometheus-operated                ClusterIP    None           <none>          9090/TCP                                2d18h
```

Récupérer le nom du bon service (dans le cadre rouge) : le nom du service svc est :

monitoring-kube-prometheus-prometheus

Utiliser ce nom dans la commande ci-dessous :

Cette commande permet de relier le service de monitoring a http sur le port 9090 ;

kubectl port-forward svc/monitoring-kube-prometheus-prometheus -n monitoring 9090

```
henri@Henri:~/labs/app-observability-lab$ kubectl port-forward svc/monitoring-kube-prometheus-prometheus -n monitoring 9090
Forwarding from 127.0.0.1:9090 -> 9090
Forwarding from [::1]:9090 -> 9090
```

ensuite faire :

👉 <http://localhost:9090>

11.2 Onglet Targets

👉 On doit voir :

python-app UP

on constate que es nos deux pods sont bien présentes e ‘‘up’’

Prometheus

Query

Alerts

Status > Target health

serviceMonitor/default/python-app/0

2 / 2 up

Endpoint	Labels	Last scrape	State
http://10.42.0.69:5000/metrics	container="python-app" endpoint="http" instance="10.42.0.69:5000" job="python-app" namespace="default" pod="python-app-7d88cbcb74-5vkpt" service="python-app"	2.175s ago 3ms	UP
http://10.42.0.70:5000/metrics	container="python-app" endpoint="http" instance="10.42.0.70:5000" job="python-app" namespace="default" pod="python-app-7d88cbcb74-sjh7" service="python-app"	14.981s ago 3ms	UP

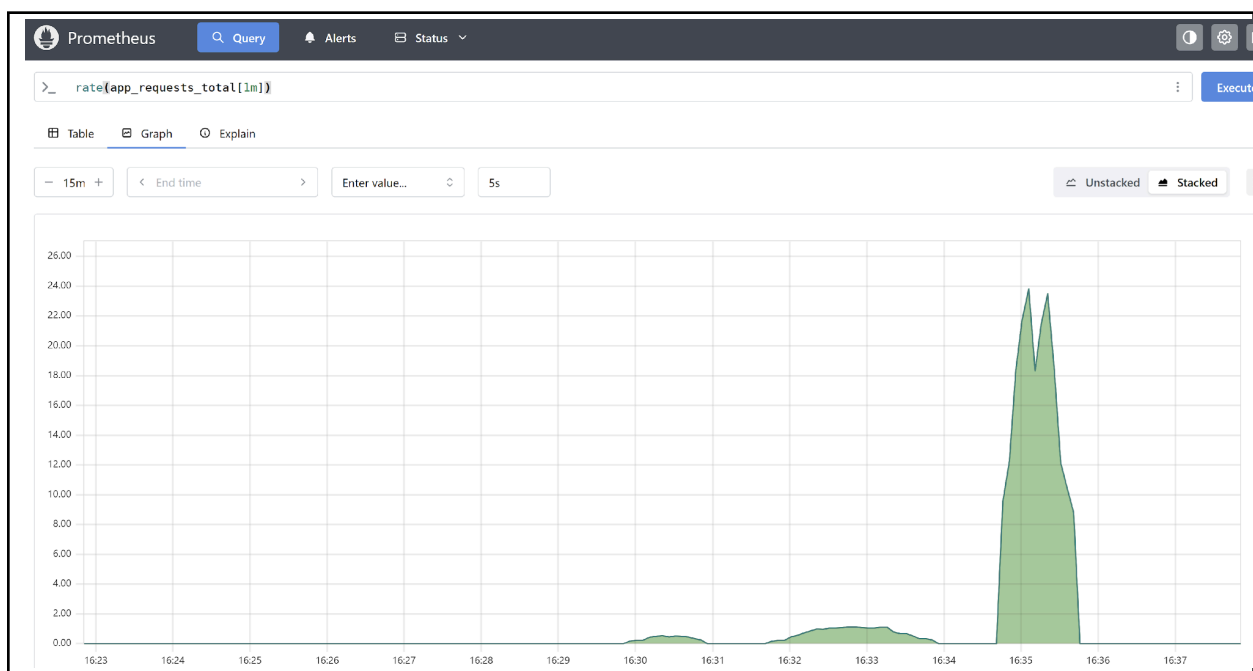
11.3. Onglet Graph

Tape : `app_requests_total` ou bien `rate(app_requests_total[1m])`

👉 On doit voir la métrique évoluer

Cette requête correspond au code python dans le code source de l'application : `app-python REQUESTS = Counter('app_requests_total', 'Total HTTP Requests')`

On peut aussi tester avec la commande :



12 STEP 11. Test GitOps

12.1 Modifier un des fichiers :

```
replicas: 2 → 4
# modifier un fichier (ex: replicas)
nano k8s/deployment.yaml
et changer replicas de 2 à 4
```

12.2 Push :

```
git add .
git commit -m "scale app to 3 replicas"
git push
```

12.3 Résultat :

- Argo CD détecte
- Kubernetes scale vers 4 pods . 2 pods supplémentaires ont été ajoutés par Argo CD : ceux dans le bloc encadré en rouge.

```
henri@Henri:~/labs/app-observability-lab$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
demo-app-58bf5cfc8d-ckd28           1/1     Running   0           20h
nginx-7769f8f85b-75j6t              1/1     Running   0           20h
nginx-7769f8f85b-8j92t              1/1     Running   0           20h
nginx-7769f8f85b-zchgx              1/1     Running   0           20h
python-app-7d88cbcb74-5vkpt          1/1     Running   0           79m
python-app-7d88cbcb74-8l6zq          1/1     Running   0           5m4s
python-app-7d88cbcb74-qc6pc          1/1     Running   0           5m4s
python-app-7d88cbcb74-ssjh7          1/1     Running   0           79m
henri@Henri:~/labs/app-observability-lab$
```

13 STEP 12. Grafana

13.1 Lancer grafana

```
kubectl port-forward svc/monitoring-grafana -n monitoring 3000:80
```

```
henri@Henri:~/labs/app-observability-lab$
henri@Henri:~/labs/app-observability-lab$ kubectl port-forward svc/monitoring-grafana -n monitoring 3000:80
Forwarding from 127.0.0.1:3000 -> 3000
Forwarding from [::1]:3000 -> 3000
```

Puis : <http://localhost:3000>

Puis : récupérer le mot de passe :

```
kubectl get secret monitoring-grafana -n monitoring -o jsonpath="{.data.admin-password}" | base64
--decode
```

Login

user: admin

password: (commande ci-dessus)

Vérifier que Grafana existe : `kubectl get svc -n monitoring | grep Grafana`

⚠ Si la commande ne marche pas

👉 liste tous les services :

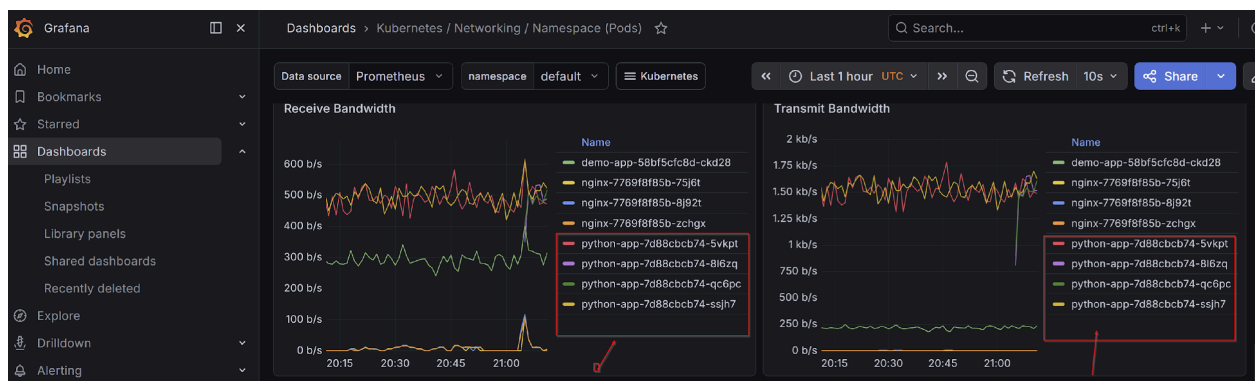
```
kubectl get svc -n monitoring
```

👉 puis adapter :

```
kubectl port-forward svc/<nom-service> -n monitoring 3000:80
```

13.2 Résultat dans grafana :

eXemple de résultats : qui montre tous les pods qui sont en fonction. J'ai encadré en rouge les 4 pods de ce lab : auparavant on avait 2 pods, maintenant on a 4 pods



14 Annexe 1 :

Prérequis d'installation

14.1 Méthode standard

Installer Prometheus + ServiceMonitor avec Helm

👉 On utilise : ➡ kube-prometheus-stack 👉 fourni par Prometheus Community

Référence officielle

👉 GitHub : ➡ <https://github.com/prometheus-community/helm-charts>

👉 Chart spécifique : ➡

<https://github.com/prometheus-community/helm-charts/tree/main/charts/kube-prometheus-stack>

- ✓ installe Prometheus
- ✓ installe Grafana
- ✓ installe Prometheus Operator
- ✓ ajoute ServiceMonitor automatiquement

👉 = solution production-like

14.1.1 STEP 1 — Installer Helm (si pas déjà fait)

👉 Helm `helm version`

14.1.2 STEP 2 — Ajouter le repo Prometheus

```
helm repo add prometheus-community  
https://prometheus-community.github.io/helm-charts  
helm repo update
```

14.1.3 STEP 3 — Créer namespace

```
kubectl create namespace monitoring
```

14.1.4 STEP 4 — Installer kube-prometheus-stack

```
helm install prometheus prometheus-community/kube-prometheus-stack -n  
monitoring
```

14.1.5 STEP 5 — Vérifier

```
kubectl get pods -n monitoring
```

👉 attendu :

```
prometheus-prometheus-0  
prometheus-grafana-xxxx  
prometheus-operator-xxxx
```

Vérifier ServiceMonitor support : `kubectl get crd | grep servicemonitor`

👉 attendu : `servicemonitors.monitoring.coreos.com`

👉 On peut maintenant utiliser ServiceMonitor

🌐 Accès Prometheus

```
kubectl port-forward svc/prometheus-kube-prometheus-prometheus -n monitoring  
9090
```

👉 <http://localhost:9090>

Accès Grafana

```
kubectl port-forward svc/prometheus-grafana -n monitoring 3000:80
```

👉 <http://localhost:3000>

Login :

user: admin

password :

```
kubectl get secret prometheus-grafana -n monitoring -o  
jsonpath="{.data.admin-password}" | base64 --decode
```

🔥 Installer ServiceMonitor (ton app)

👉 Ensuite seulement :

```
kubectl apply -f servicemonitor.yaml
```

⚠️ POINT CRITIQUE (très important)

Dans ton `servicemonitor.yaml` :

```
metadata:
```

```
  labels:
```

```
    release: prometheus
```

👉 doit matcher le nom Helm :

```
helm install prometheus ...
```

Résumé

Helm → kube-prometheus-stack → Prometheus + Grafana + ServiceMonitor OK

14.2 Alternative (moins recommandée)

👉 Prometheus seul (sans operator)

➡ https://prometheus.io/docs/prometheus/latest/getting_started/

- ❌ pas de ServiceMonitor
- ❌ config manuelle
- ❌ pas utilisé en prod moderne